

EECS 570

Lecture 12

# Weaker Consistency Models

Fall 2025

Prof. Satish Narayanasamy

<http://www.eecs.umich.edu/courses/eecs570/>

Slides developed in part by Profs. Adve, Falsafi, Hill, Lebeck, Martin, Narayanasamy, Nowatzyk, Reinhardt, Singh, Smith, Torrellas and Wenisch.

# What states are infeasible under SC?

| Core 0                     | Core 1                       |
|----------------------------|------------------------------|
| (i1) x = 1;<br>(i2) y = 1; | (i3) r1 = y;<br>(i4) r2 = x; |
|                            |                              |

---

|                    |                        |
|--------------------|------------------------|
| A=0                | flag=0                 |
| <u>Processor 0</u> | <u>Processor 1</u>     |
| A=1;               | while (!flag); // spin |
| flag=1;            | print A;               |

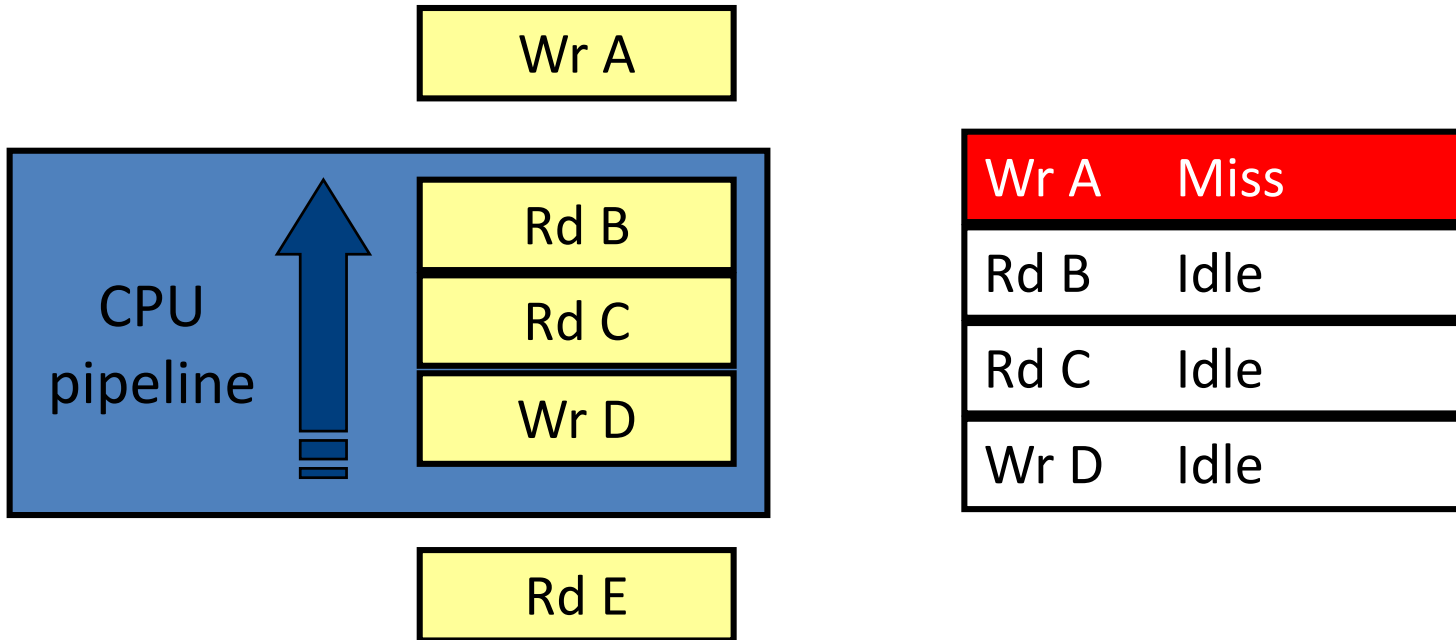
---

|           |               |
|-----------|---------------|
| A=0       | B=0           |
| <u>P1</u> | <u>P2</u>     |
| A=1;      | while (A==0); |
|           | B = 1;        |

# Review: Problems with SC

- Difficult to implement efficiently in hardware
  - ▣ Straight-forward implementations:
    - No concurrency among memory access
    - Strict ordering of memory accesses at each node
    - Essentially precludes out-of-order CPUs
- Unnecessarily restrictive
  - ▣ Most parallel programs won't notice out-of-order accesses
- Conflicts with latency hiding techniques

# Execution in strict SC



- Miss on Wr A stalls all unrelated accesses

Memory accesses issue one-at-a-time

# Hardware Relaxed Consistency Models

# Sun's "Total Store Order" (TSO)

## TSO: Hardware Interface

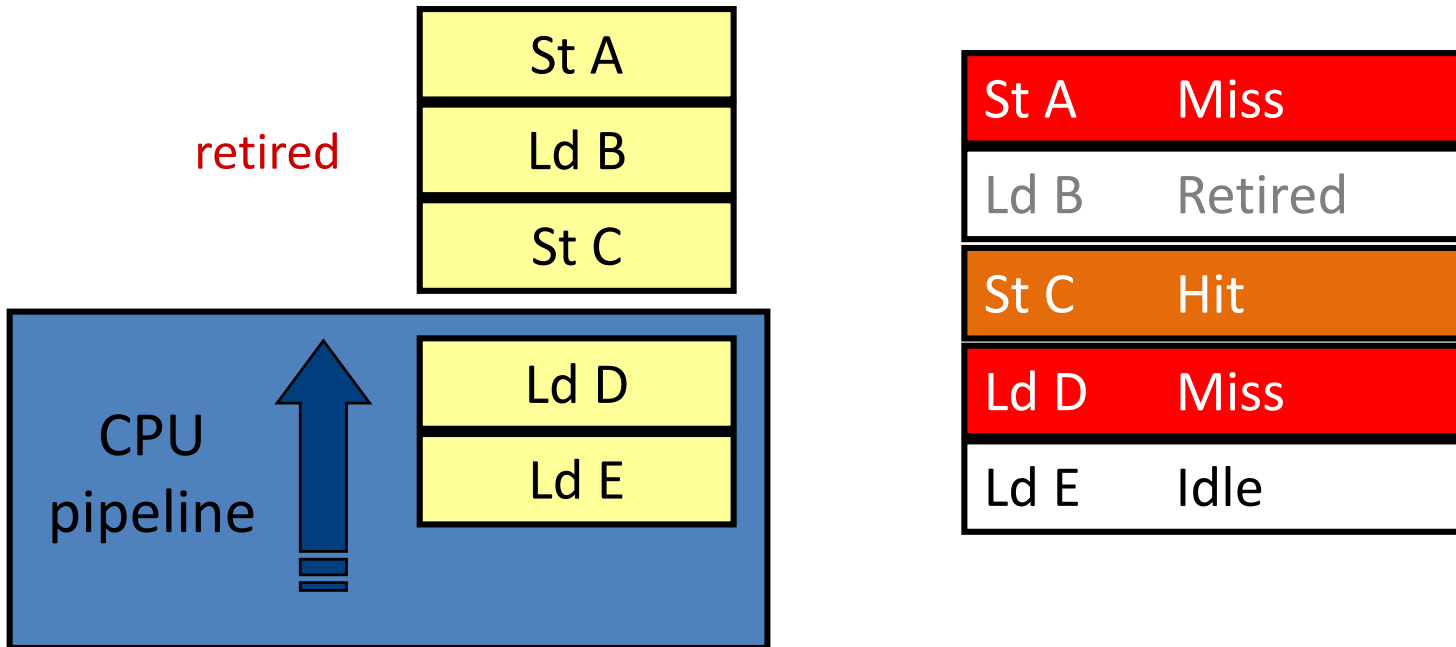
*(Total Store Order — intuition)*

- **Goal:** Let **loads go ahead of pending stores** to improve performance
- Stores are **buffered** instead of being written to memory immediately
- The processor **ensures program order for stores**, but **allows loads to bypass** them safely
- To preserve correctness, **barriers (STBAR)** or **RMW ops** are used to re-establish order when needed

# TSO: HW Perspective

- Enables a **FIFO-ordered, non-coalescing store buffer**
  - ❑ Typically maintains stores at word-granularity
  - ❑ Loads search buffer for matching store(s)
    - Some ISAs must deal with merging partial load matches
  - ❑ Coalescing only allowed among adjacent stores to same block
  - ❑ Must force buffer to drain on RMW and STBAR (type of fence)
  - ❑ Often, this is implemented in same HW structure as (speculative) store queue
- Can hide store latency!
  - ❑ But, store buffer may need to be quite big
    - Stores that will be cache hits remain buffered behind misses
  - ❑ Associative search limits scalability
    - E.g., certainly no more than 64 entries

# Execution in TSO



- Stores misses do not stall retirement
  - ❑ St A, St C, Ld D misses overlapped
  - ❑ Ld B retired without waiting for St A to fill
  - ❑ St C consumes space in store buffer even though it will hit

**TSO hides store miss latency**



# TSO: Compiler's Perspective

- Compiler may now hoist loads across stores
  - ❑ Still can't reorder stores or move loads across loads
  - ❑ **Not clear how helpful this is in practice...**
  - ❑ ...Recent results from Prof. Satish's group:
    - 5-30% perf. gap vs. compiler that preserves SC [PLDI'11]
- No new crazy memory barriers to emit
  - ❑ Library/OS writers need to use a little bit of caution; use RMWs instead of loads/stores for synchronization
  - ❑ TSO-unsafe code is rare enough that it can be the programmer's problem
- No need to invoke “undefined behavior” to avoid onerous implementation requirements

# Dekker's Algorithm

- Mutually exclusive access to a critical region
  - Works as advertised under sequential consistency

/\* initial A = B = 0 \*/

P1

A = 1;

if (B != 0) goto retry;

/\* enter critical section \*/

P2

B=1;

if (A != 0) goto retry;

/\* enter critical section \*/

# TSO: Programmer's Perspective

- Can occasionally lead to astonishing behavior changes
  - ❑ E.g., Dekker's algorithm doesn't work
  - ❑ ISAs provide an STBAR (store barrier) to manually force order
    - Semantics – store buffer must be empty before memory operations after STBAR may be executed
  - ❑ Can also enforce order by replacing stores with RMWs
- But, the key case, where sync is done with locks, simply works
  - ❑ Lock acquires are RMW operations  $\Rightarrow$   
they force order for preceding/succeeding loads/stores
    - Load semantics of RMW imply load-load orderings
    - Ditto for store semantics
  - ❑ Lock release is a store operation  $\Rightarrow$   
it must be performed after critical section is done

# TSO Variants

- Differ in their notions of write atomicity
- IBM 370 was the same as TSO except that loads could not read from the store buffer early
- Consider:

| Core 0                                          | Core 1                                      |
|-------------------------------------------------|---------------------------------------------|
| (i1) x = 1;<br>(i2) r1 = x;<br>(i3) r2 = y;     | (i4) y = 1;<br>(i5) r3 = y;<br>(i6) r4 = x; |
| IBM 370 Forbids: r1 = 1, r2 = 0, r3 = 1, r4 = 0 |                                             |

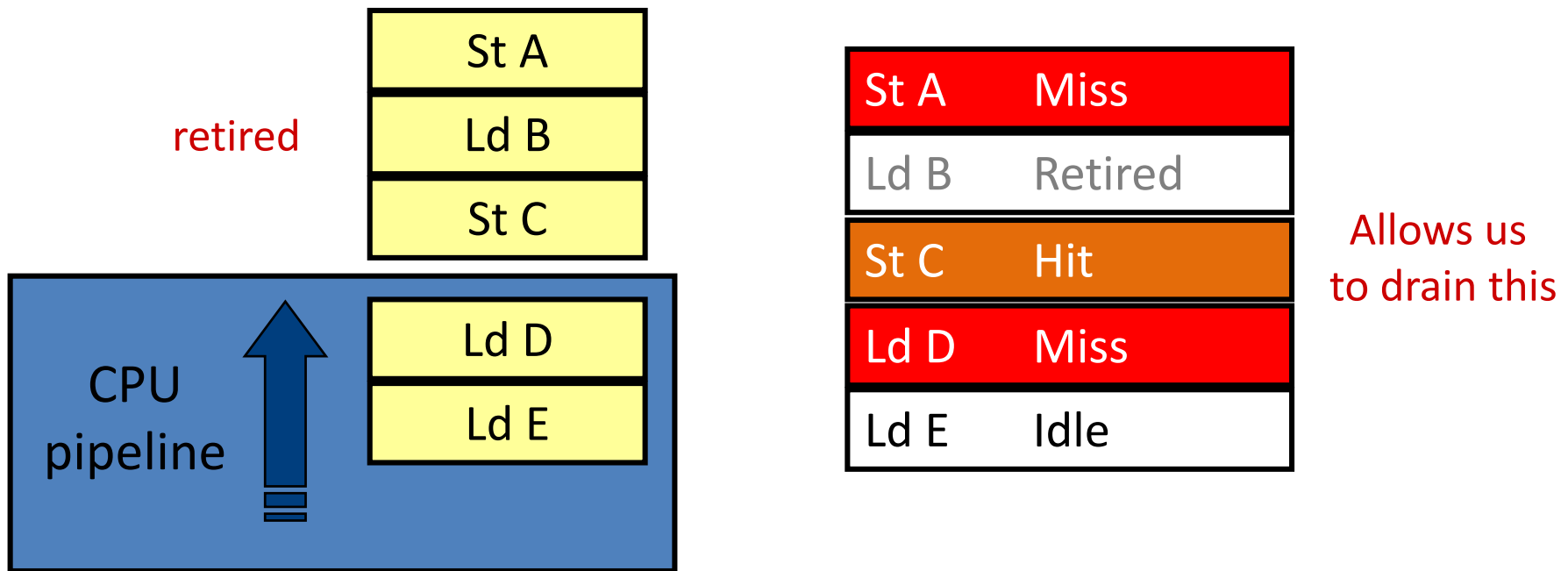
# Processor Consistency (PC)

- [Goodman 1989]
- Basically TSO with the relaxation of store atomicity
- Consider the IRIW litmus test:

| Core 0                                           | Core 1 | Core 2             | Core 3             |
|--------------------------------------------------|--------|--------------------|--------------------|
| x = 1;                                           | y = 1; | r1 = x;<br>r2 = y; | r3 = y;<br>r4 = x; |
| Allowed under PC: r1 = 1, r2 = 0, r3 = 1, r4 = 0 |        |                    |                    |

# Relaxing Write-to-Write Order

- Motivation: Coalescing store buffers & early drain



- Allows writes to coalesce in SB & drain early

# Sun's "Partial Store Order" (PSO)

- Formal requirements [v8 architecture manual]:
  - ❑ **Order** - There exists a partial memory order ( $<_M$ ) and it is total for all operations with store semantics
  - ❑ **Atomicity** - Atomic read-modify-writes do not allow a store between the read and write parts
  - ❑ **Termination** - All stores are performed in finite time
  - ❑ **Value** – Loads return the most recent value w.r.t. memory order ( $<_M$ ) and w.r.t. local program order ( $<_p$ )
  - ❑ **LoadOP** – Loads are blocking
  - ❑ **StoreStore** – Stores are ordered only if they are separated by a *membar* (memory barrier) instruction
  - ❑ **StoreStoreEq** – Stores to the same address are ordered

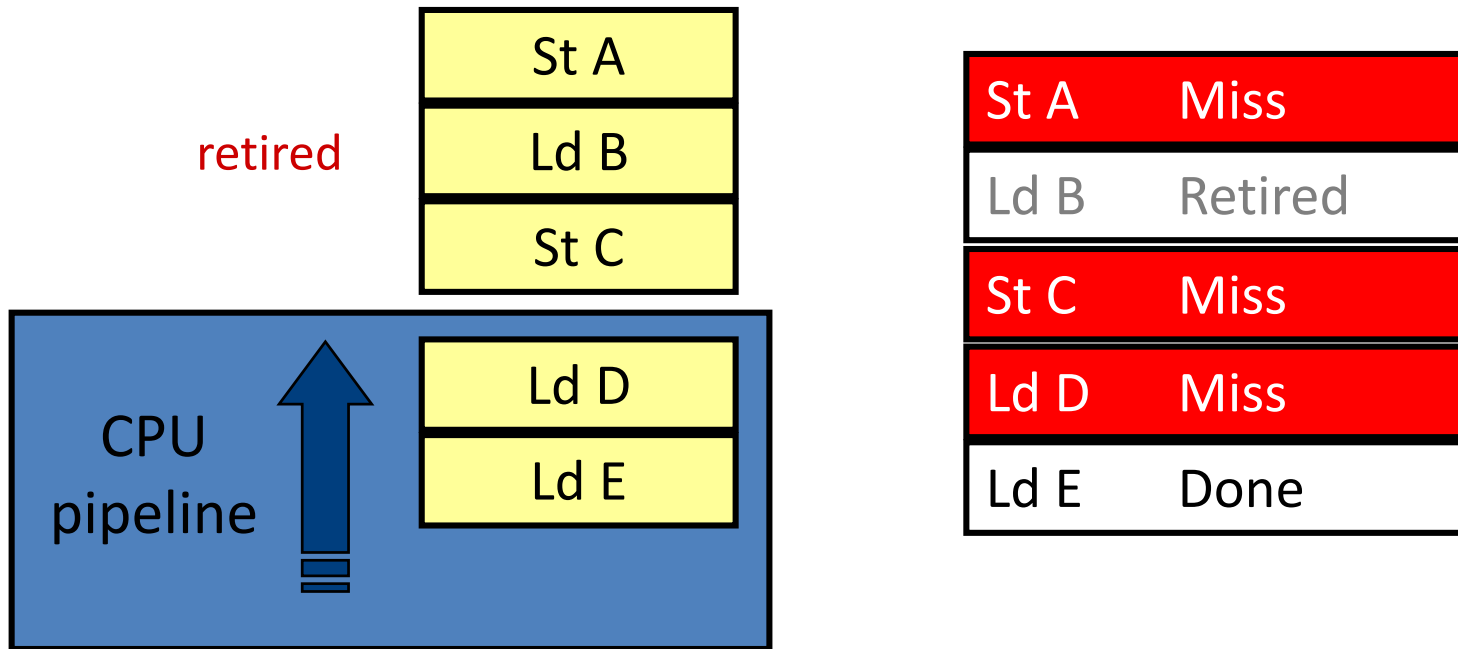
# PSO: Compiler/HW Perspective

- Allows an **unordered, coalescing post-retirement store buffer**
  - ❑ Can now use a cache-block-grain set-associative structure
  - ❑ Store misses leave store buffer upon cache fill
  - ❑ Loads search buffer for matching store(s)
  - ❑ Must force buffer to drain on RMW and STBAR
- Much more efficient store buffer
- But, still doesn't allow out-of-order loads
  - ❑ No OoO execution (without speculation)
  - ❑ Compiler's hands are still tied



# RMO: Relaxing all Order

- Motivation: Out-of-order execution & multiple load misses



- Now Ld E can complete even though earlier insn aren't done

# Two ISA Approaches to enforce order

- Approach 1: Using explicit “fence” (aka memory barrier)
  - ▣ Sun’s Relaxed Memory Order (RMO), Alpha, PowerPC, ARM

Ld, St, ...

L L S S

Fence

↓↓↓↓

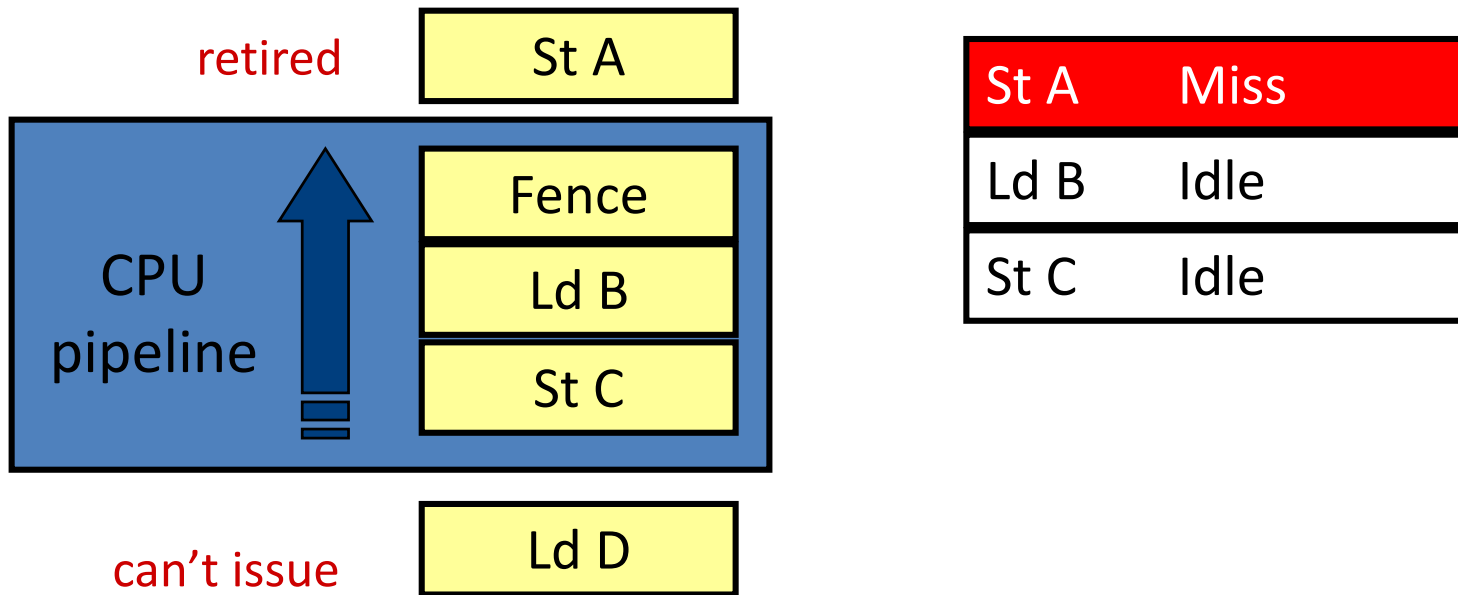
Enforces order if bit is set

L S L S

Ld, St, ...

- Approach 2: Annotate loads/stores that do synchronization
  - ▣ RMW – atomic instructions

# Execution in RMO w/ fence



- “Fence” indicates ordering affects correctness
  - ❑ Retirement stalls at Fence
  - ❑ Typically, accesses after fence don't issue

# RMO: Compiler's Perspective

- **Sweet, sweet freedom!**
  - ❑ Compiler may freely re-order between fences
  - ❑ Also across fences if the fence allows it (partial fences)
  - ❑ Note: still can't reorder stores to same address
  - ❑ Programmer's problem if the fences are wrong...

# RMO: HW Perspective

- Unordered, coalescing post-retirement store buffer
  - ▣ Just like PSO
- Out-of-order execution!
  - ▣ Need a standard uniprocessor store queue
  - ▣ Fence instruction implementation
    - Easy – stall at issue
    - Hard – retire to store buffer and track precise constraints

# Data Races

A program has a **data race** if it has an execution in which two accesses to the **same address** on **different threads** where **at least one is a write** and **at least one is not a synchronization access** are not ordered by synchronization accesses

Note: The definition of races can vary depending on the context.

Core 0

A: x = 1;

B: y = 1;

Core 1

C: r1 = y;

D: z = 1;

# Data Race Free Memory Model

A program is **data-race-free** if all data races are appropriately annotated (`volatile/atomic`)

## DRFO

[Adve & Hill 1990]

SC behavior for data-race-free programs,  
**weak** or **no** semantics otherwise

Java Memory Model  
(JMM)

[Manson et al. 2005]

C++0x Memory Model

[Boehm & Adve 2008]

# DRFo-compliant Program

```
    X* x = null;  
    atomic bool flag = false;
```



```
A: x = new X();  
B: flag = true;
```



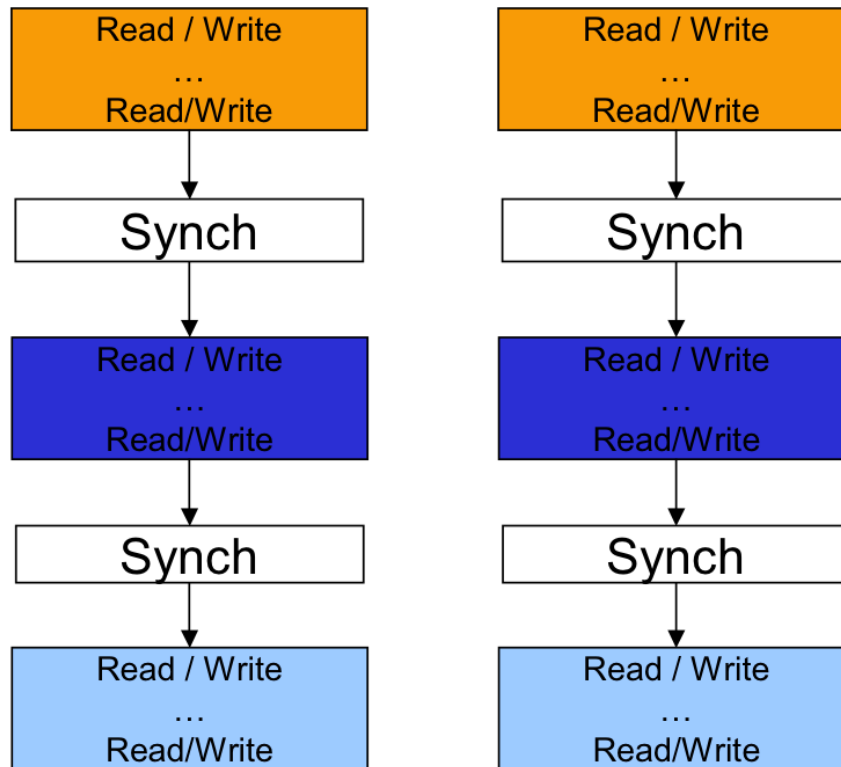
```
C: while(!flag);  
D: x->f++;
```

- DRFo guarantees SC  
.... only if data-race-free (all *unsafe* accesses are annotated)
- What if there is one data-race?  
.... all bets are off (e.g., compiler can output an empty binary!)



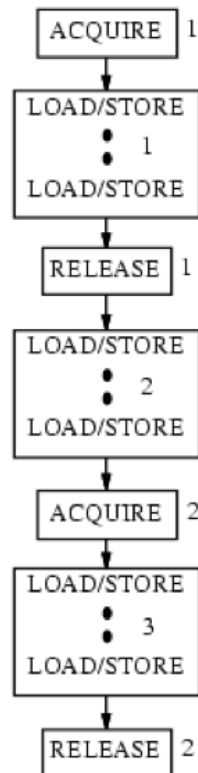
# Weak Ordering (WO)

- [Dubois et al. ISCA 1986]
- Loads and stores can be labeled as “sync”
  - ❑ No reordering allowed across sync instructions

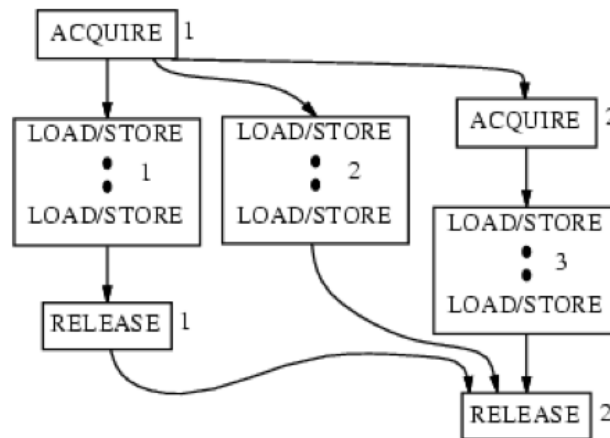


# Release Consistency (RC)

- Specialize loads and stores as “acquires” and “releases”
  - Load at top of critical section is an acquire
  - Store at bottom of critical section is a release
  - Allows reordering **into** critical sections, but not out



Weak Consistency (WCsc)



Release Consistency (RCpc)

u  
↓  
v  
v cannot perform with respect to  
any other processor until u is  
performed

# WO/RC: Programmer Perspective

- A new way of thinking: **programmer-centric memory models**
  - ❑ If you annotate syncs correctly, your program will behave like SC
    - WHY?
  - ❑ E.g., Data-Race-Free-0 (DRF0) [Adve and Hill ISCA 1990]
    - Accesses are either normal or sync
    - Sync accesses are sequentially consistent, and order normal accesses
    - Data races among normal accesses are prohibited
    - **DRF0 programs appear as if they ran under SC**
    - Similar idea in [Gharachorloo et al. ISCA 1990]: “Properly-[Labelled]” Programs
  - ❑ SC-for-DRF forms the basis for programming language memory models

# The Ramifications of SC-for-DRF

- Hardware can freely reorder instructions in between acquire and release operations
  - ❑ Programmers can't tell!
  - ❑ DRF executions appears to honor SC
- Compilers can freely reorder code in between acquire and release operations
  - ❑ Again, programmers can't tell!
  - ❑ DRF executions appears to honor SC